TC2037 Implementation of Computational Methods

Activity 6.2 Distributed Programming in Erlang

This team project will give you the opportunity to practice and become familiar with the Erlang programming language. Projects with fewer than the specified number of team members **will not be accepted**. Any issues regarding this should be discussed with the instructor as soon as possible.

You must document your code internally using Erlang comments. These comments must include your student IDs and names in the file, as well as descriptions of functions, process creation and communication protocols, and auxiliary functions. Failure to adequately document your code will result in severe penalties.

Description of the Evidence:

The objective of this activity is to design and implement a **distributed taxi rental system for travel to an airport**. This system must be able to manage passenger requests, allocate available taxis appropriately, and maintain up-to-date service information.

SYSTEM OPERATION

You must program a distributed system that involves the following 3 types of entities:

- **Travelers** : Customers who request a taxi service to the airport.
- **Taxis** : Vehicles registered in the system, available to make trips (**processes**).
- **Taxi Dispatch Center**: Server that manages taxi records, travel requests and the assignment of taxis to travelers (**process**).

The **travelers** must request a taxi from the Taxi Central. To do so, they must use the command `traveler:request_taxi(Traveler,Origin)` , where **Traveler** must be an atom (name) that uniquely identifies the traveler and **Origin** is the pickup location (a tuple {X,Y}). Different names must be specified for the traveler to be accepted.

The Taxi Dispatch Center will respond by assigning a taxi if one is available and accepts the trip, or with a message indicating unavailability. The Dispatch Center will generate a unique trip ID for each accepted request and will respond to the traveler with the assigned taxi ID and the trip ID. If multiple taxis are available, the Dispatch Center should attempt to assign them based on the traveler's distance, from the closest as the to the farthest, using the Euclidian distance.

The traveler can cancel a taxi request using the command **`traveler:cancel_taxi(Traveler).`** If the traveler has a taxi request for which the service has not yet started, the dispatch center must cancel it.

Taxis Taxis must be registered at the Central using the command **`taxi:register_taxi(TaxiId,InitialLocation)`**, where **TaxiId** is a unique mnemonic identifier for the taxi, and **InitialLocation** is the taxi's location (a tuple {X,Y}). Registering a taxi marks it as active and creates a process that monitors its status (available/occupied) and location. A taxi can update its location using the command **`taxi:current_location(TaxiId,Location)`**.

In this way, when a traveler requests a taxi, the Taxi Dispatch Center will interact (via messages) with the processes of the active taxis to determine which one can be assigned and change its status. The taxi accepts a trip with the command **`taxi:accept_trip(TripId)`** or rejects it with the command **`taxi:reject_trip(TripId)`**. If a taxi rejects a trip, the dispatch center must try the next taxi closest to the traveler.

An active taxi can be removed by terminating its process with the command **`taxi:remove_taxi(TaxiId)`**. To be removed, the taxi must be in an available state. The status and location of a taxi can be checked using **`taxi:consult_taxi(TaxiId)`**.

The taxi must notify the dispatch center when it starts a transport service using the command **`taxi:service_started(TaxiId).`** If that taxi was indeed assigned to a trip, it should remain **occupied** at the trip's origin location. The taxi must notify the dispatch center when it finishes a service using the command **`taxi:service_completed(TaxiId)`**. If that taxi was indeed occupied with a service, it should become available at the airport location and complete the passenger's process.

The **Taxi Dispatch Center** should be managed as a process that centralizes information on registered taxis, active trips, and a history of completed trips. For taxis, only a list with their names (IDs) and PIDs for those with an active trip should be stored. For trips, a counter should be kept tracking the number of trips, along with a list containing the historical record of each completed trip (trip ID, taxi ID, passenger, origin). The trip counter increments with each assignment.

The Taxi Dispatch process is created with the command **`center:open_center(AirportLocation)`** and terminated with the command **`center:close_center()`**. The Taxi Dispatch Center process must be registered

with the name **center** so that passengers and taxi dispatchers do not need to know its PID. The airport location must be a tuple {X,Y} with its coordinates. This process must implement communication protocols for passengers requesting taxis and for taxi creation, trip acceptance, status/location updates, and termination. Additionally, the following information must be displayed in a formatted manner: the list of active taxis with the command **`center:taxi_list()`**, the list of current passengers with the command **`center:travelers_list()`**, and the list of total completed trips with the command **`center:completed_trips()`**. Each list must contain all relevant information.

The Taxi Dispatch Center process, as well as the processes for all taxis and passengers, **can reside on the same or different nodes**. It's worth noting that **if the Taxi Dispatch process terminates**, all taxi processes must also terminate automatically. However, if a taxi's process terminates, only its entry in the active taxi list should be removed.

Since this is a prototype of a distributed system, it must include deployment code **that clearly and thoroughly describes what is happening at each node in the system**. For example, when a traveler requests a taxi, the system should display something like "Sends: <traveler_name> requests taxi from <origin>" before pausing to wait for a response from the dispatch center. Similarly, the dispatch center process, upon receiving the traveler's request, should display "Receives: taxi request from <traveler_name>" before processing the request. This pattern should apply to any message exchange between processes. These displays will appear on the various nodes where the processes are running.

Finally, **you MUST** include within the same program, using Erlang comments, information describing **the steps to be performed on each Erlang node and the specific command sequence to be executed to test all the programmed features** included in your distributed system. This must include all the details for creating the nodes and processes, as well as using all their implemented interface functions, so that all the programmed features are illustrated. Include images or listings showing the expected output of the commands described on each node. Failure to document the use of your system will be severely penalized.

SUCCESS! !